Name: Keerthi

Program: Azure Data Engineering

1. Introduction

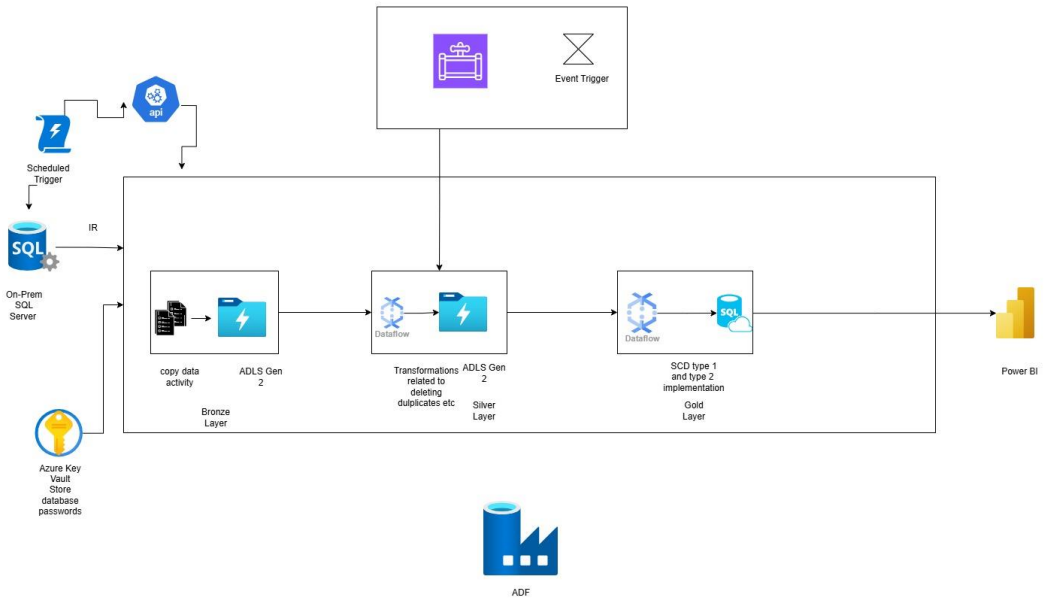
This project demonstrates the implementation of an end-to-end Azure Data Engineering pipeline using Azure Data Factory (ADF), Azure Data Lake Storage Gen2 (ADLS Gen2), Azure SQL Database, and Data Flows. The objective of this project is to design, build, and orchestrate scalable data pipelines capable of ingesting, transforming, and loading data from multiple sources into a structured target system.

2. Architecture Overview

The architecture is designed to handle data ingestion from APIs and On-Premises sources, store raw data in ADLS Gen2, perform transformations using Mapping Data Flows, and load processed data into Azure SQL Database.

- Main Components:
- Azure Data Factory (ADF) – Orchestration and pipeline management
- Azure Data Lake Storage Gen2 – Raw and staged data storage
- Mapping Data Flows – Data transformation logic
- Azure SQL Database – Final structured data sink

ARCHITECTURE:

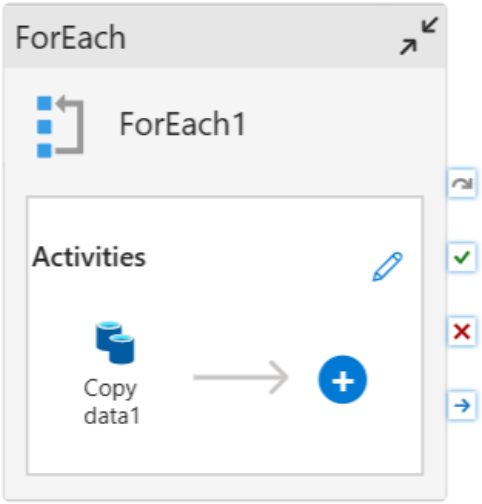


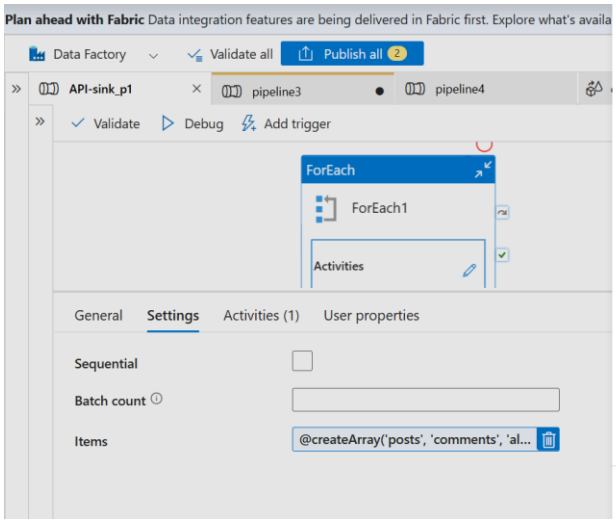
3. API to ADLS Gen2 Pipeline

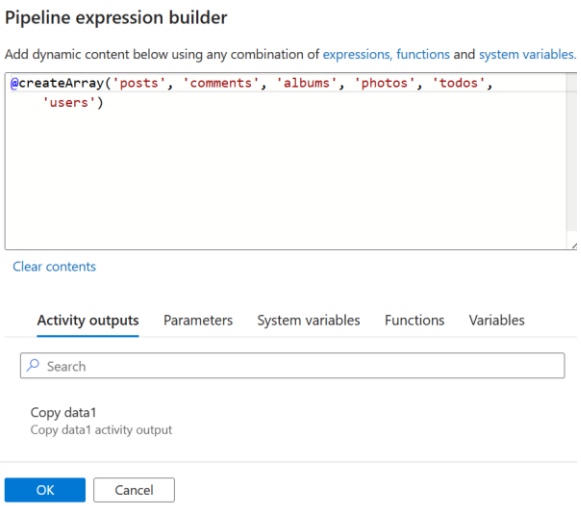
This pipeline extracts data from a REST API source and loads it into Azure Data Lake Storage Gen2.

1. Key Steps:
2. Lookup Activity – Retrieves dynamic parameters.
3. ForEach Activity – Iterates through API response values.
4. Copy Activity – Copies API data into ADLS Gen2 in CSV format.

Dynamic Content Used: @activity('Lookup1').output.value inside the ForEach loop to iterate through retrieved records.







General | Source | Sink | Mapping | Settings | User properties

Source dataset * RestResource1 Open New Preview data Learn more

Dataset properties

Name	Value
tablename	@item()

Request method GET

Request timeout 00:01:40

Request interval (ms) 10

Additional headers New

General | Source | Sink | Mapping | Settings | User properties

Sink dataset * AzureSqlTable1 Open New Learn more

Dataset properties

Name	Value
tablename	@item()

Write behavior Insert Upsert Stored procedure

Bulk insert table lock Yes No

Table option Use existing Auto create table

Pre-copy script

Azure SQL Database

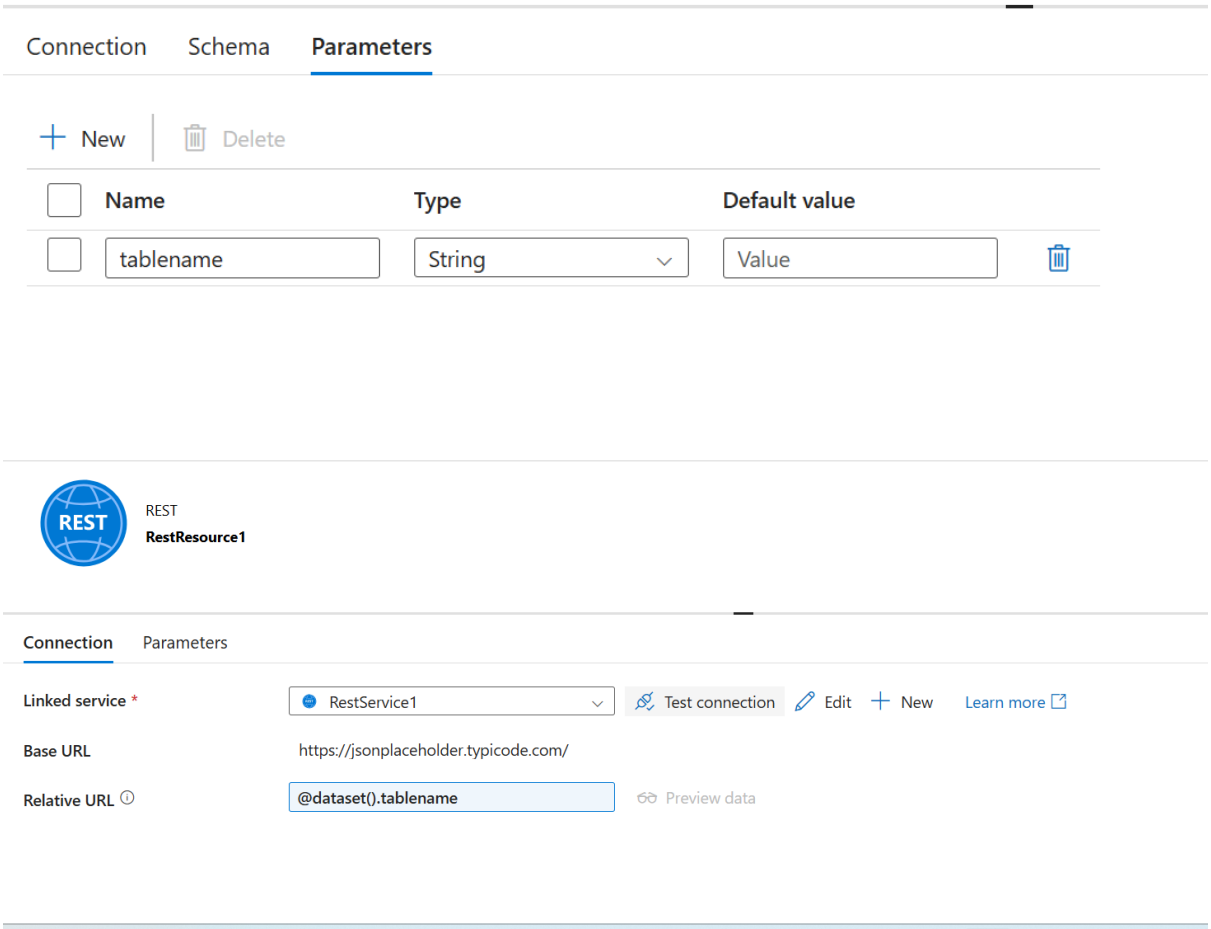
AzureSqlTable1

Connection | Schema | Parameters

Linked service * AzureSqlDatabase1 Test connection Edit New Learn more

Table dbo . @dataset().tablename Preview data

Enter manually

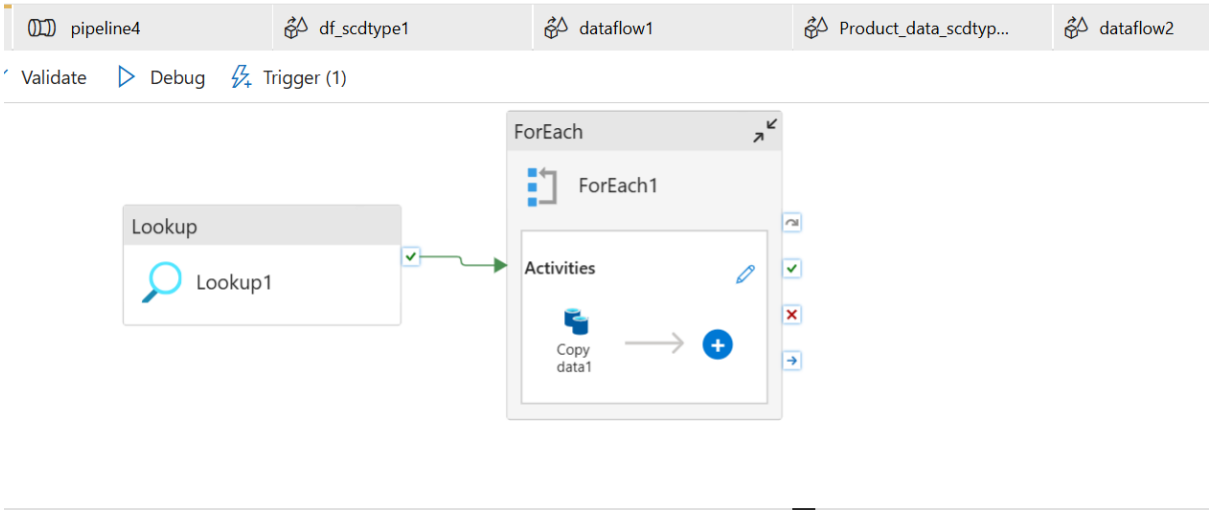


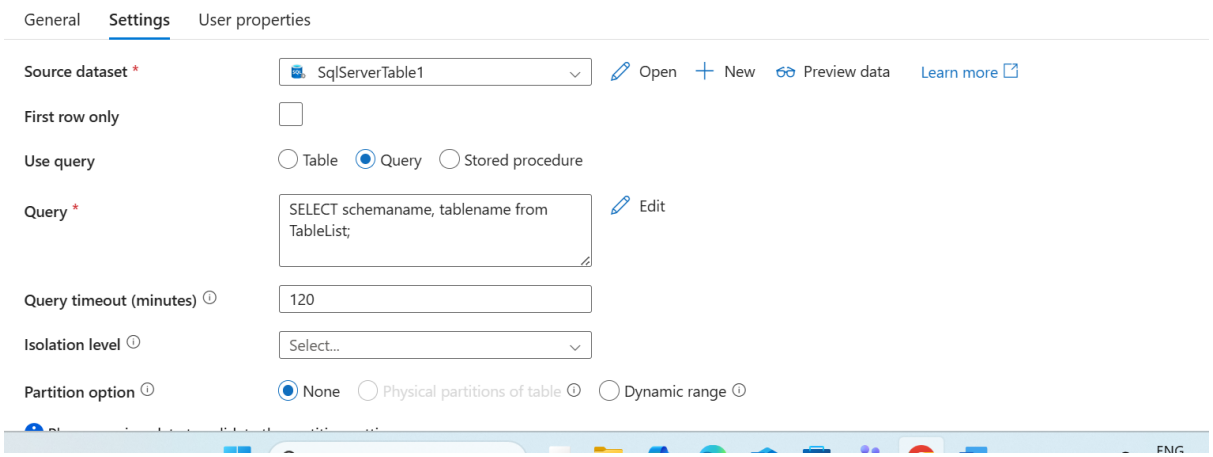
4. On-Premises to Sink Pipeline

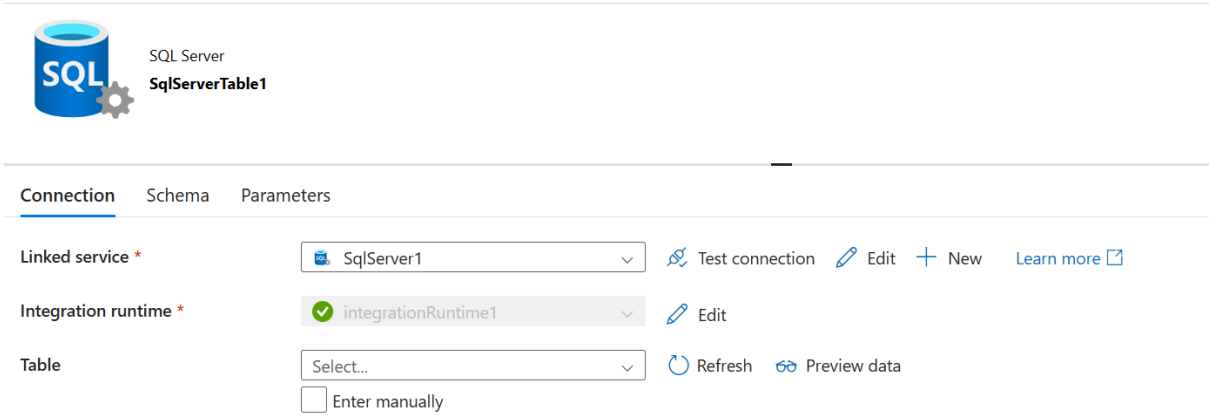
This pipeline ingests data from an On-Premises data source using a Self-Hosted Integration Runtime and loads it into the designated sink (Azure SQL Database).

- Key Configurations:
- Self-Hosted Integration Runtime setup
- Linked Service configuration for secure connectivity
- Dataset parameterization for dynamic file handling

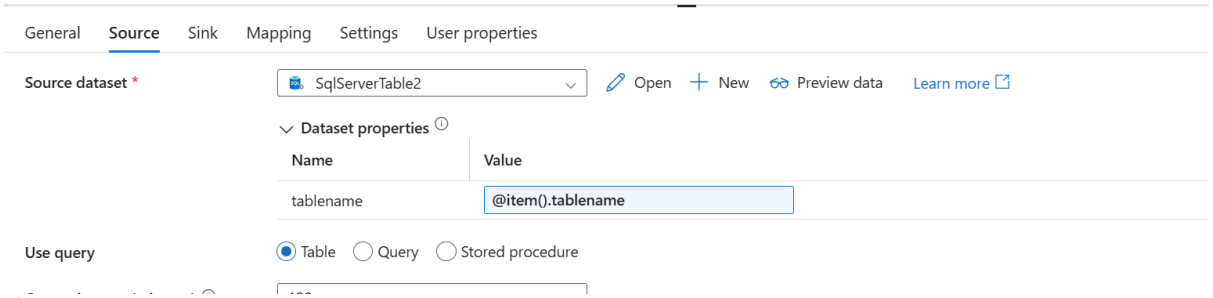
OnPrem to Sink:

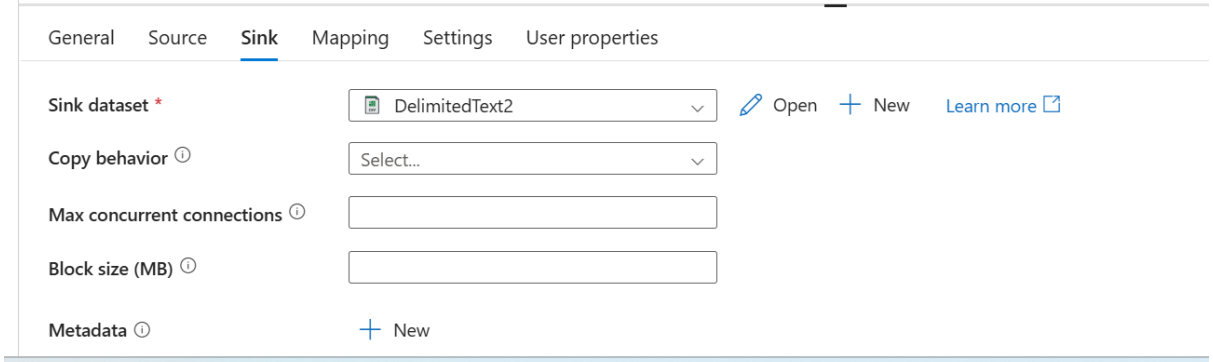


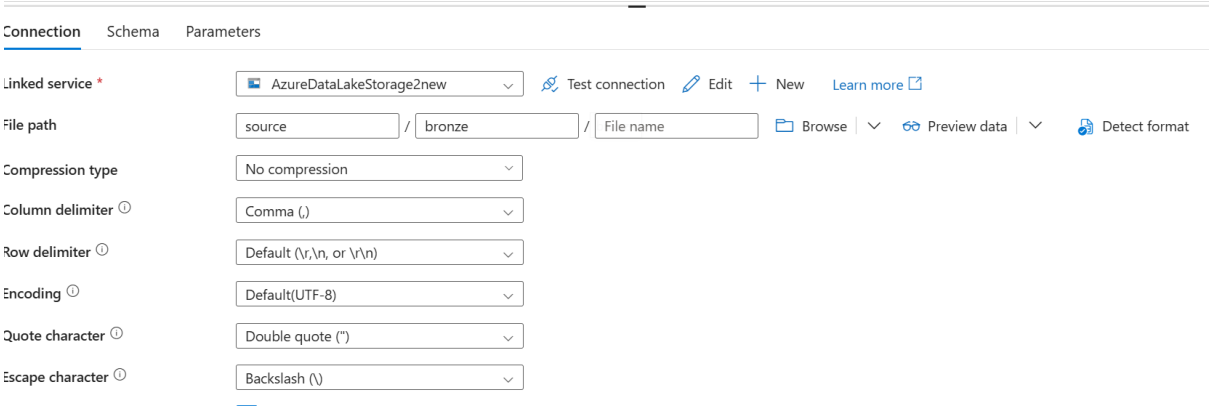




Command inside for loop: @activity('Lookup1').output.value





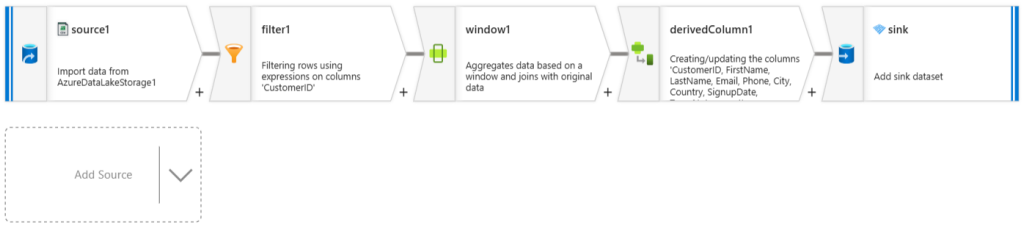


5. DataFlow1 – Transformation Logic

DataFlow1 is responsible for performing transformation operations such as filtering, derived columns, data type casting, and aggregations before loading into the sink.

- Transformations Applied:
- Column mapping and selection
- Data cleaning and null handling
- Derived column transformations
- Conditional split for business rules

Dataflow1:



Source settings | Source options | Projection | Optimize | Inspect | Data preview

Output stream name * source1 Learn more

Description Import data from AzureDataLakeStorage1 Reset

Source type * Dataset | Inline

Inline dataset type * DelimitedText

Linked service * AzureDataLakeStorage1 Test connection Edit + New

Skip line count

Source settings | Source options | Projection | Optimize | Inspect | Data preview

File settings

File mode File Wildcard

File path * sink / bronze / dbo.customer.csv Browse

Allow no files found

Change data capture

Compression type No compression

Encoding Default(UTF-8)

Column delimiter Comma (,)

Filter settings | Optimize | Inspect | Data preview

Output stream name * filter1 Learn more

Description Filtering rows using expressions on columns 'CustomerID' Reset

Incoming stream * source1

Filter on * !isNull(CustomerID)

Window settings | Optimize | Inspect | Data preview

Output stream name * window1 Help Learn more

Description Aggregates data based on a window and joins with original data Reset

Incoming stream * filter1

1. Over | 2. Sort | 3. Range by | 4. Window columns

filter1's column	Name as
12s CustomerID	CustomerID

Incoming stream * filter1

1. Over | 2. Sort | 3. Range by | 4. Window columns

Add | Clone | Delete | Open expression builder

Column	Expression
count	rowNumber()

Derived column's settings | Optimize | Inspect | Data preview

Previou

Output stream name * derivedColumn1 Learn more

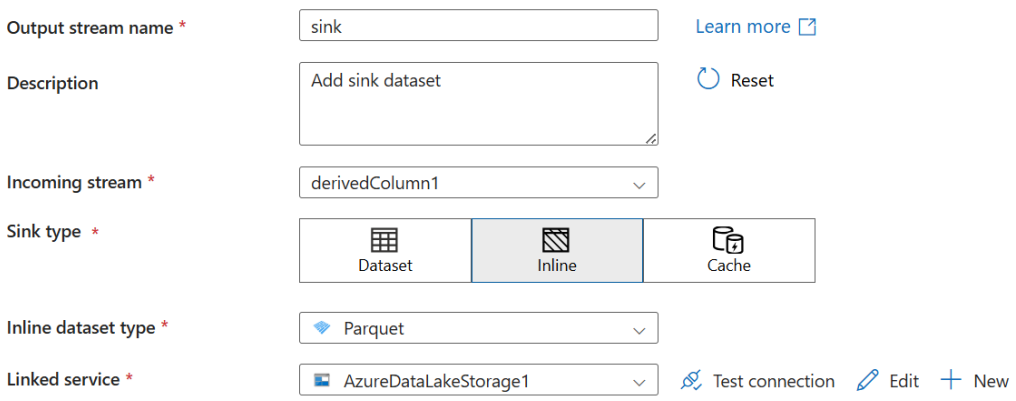
Description Creating/updating the columns 'CustomerID, FirstName, LastName, Email, Phone, City, Country, SignupDate, Reset

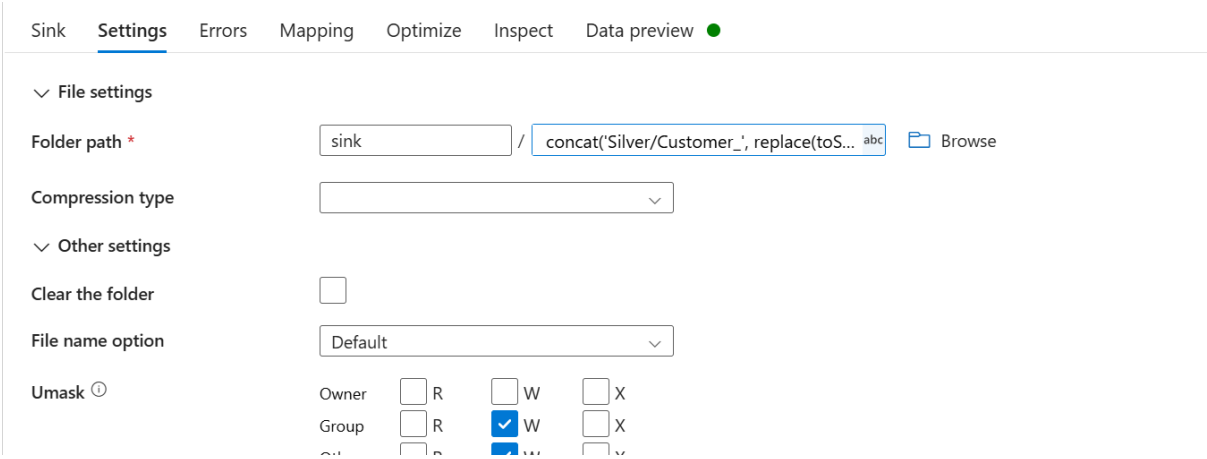
Incoming stream * window1

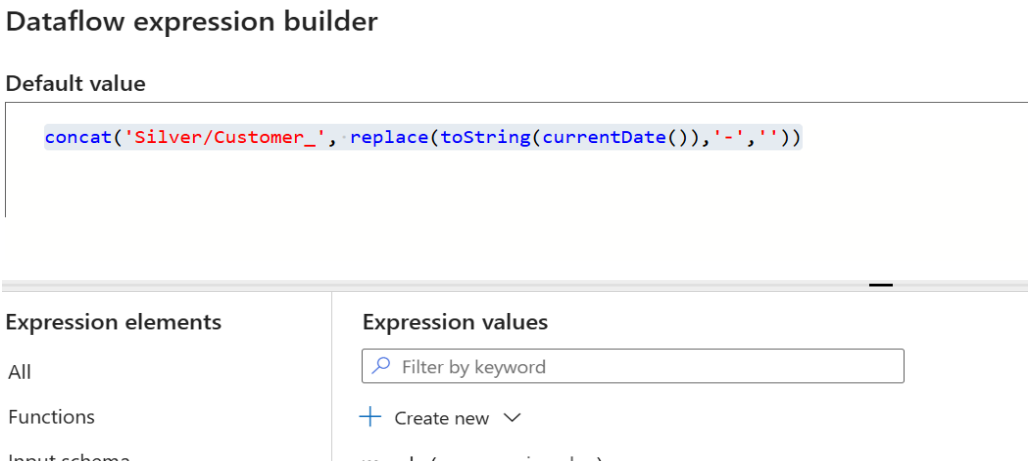
Add | Clone | Delete | Open expression builder

Columns *

Column	Expression
count	count==1
Phone	replace(Phone, 'NaN','000000000')





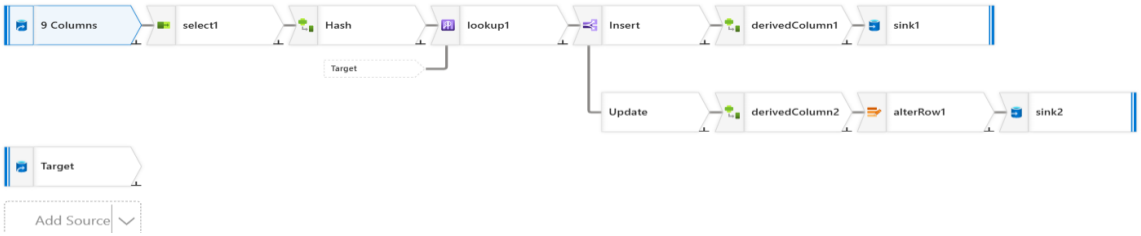


6. SCD Type 1 Dataflow Implementation

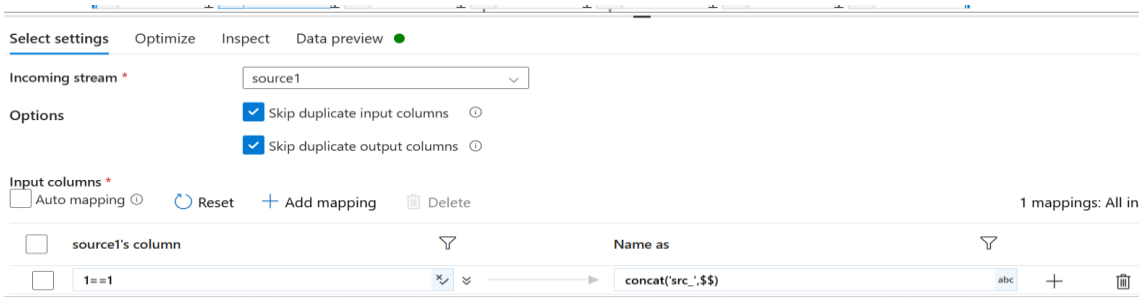
Slowly Changing Dimension (SCD) Type 1 logic is implemented to overwrite existing records in the target table when matching keys are found.

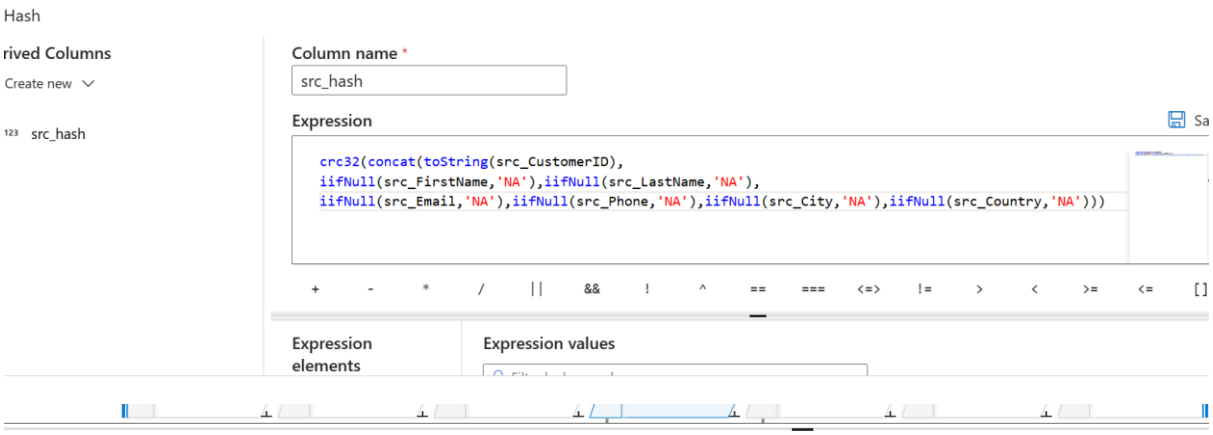
5. Steps in SCD Type 1:
6. Source – Incoming dataset
7. Select – Choose required columns
8. Alter Row – Define insert/update conditions
9. Sink – Azure SQL Database with upsert enabled

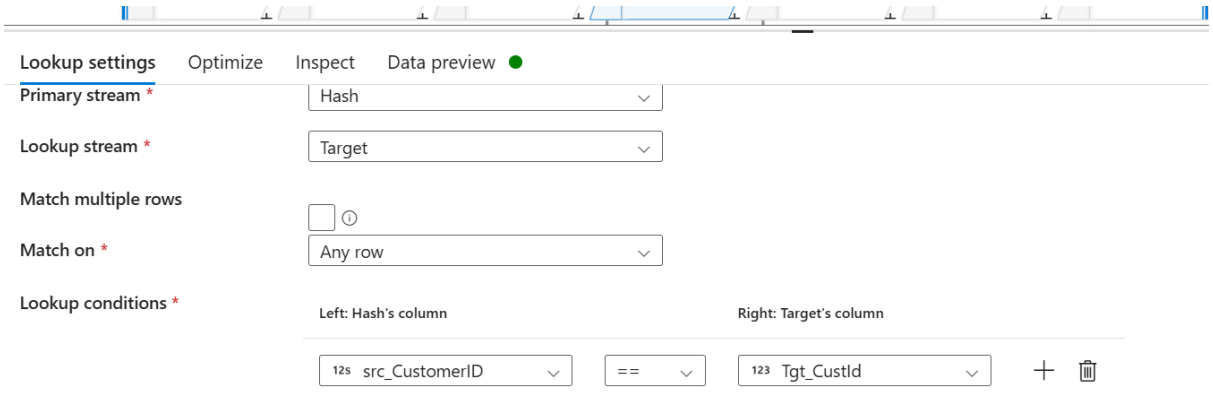
SDCTYPE1 Dataflow:

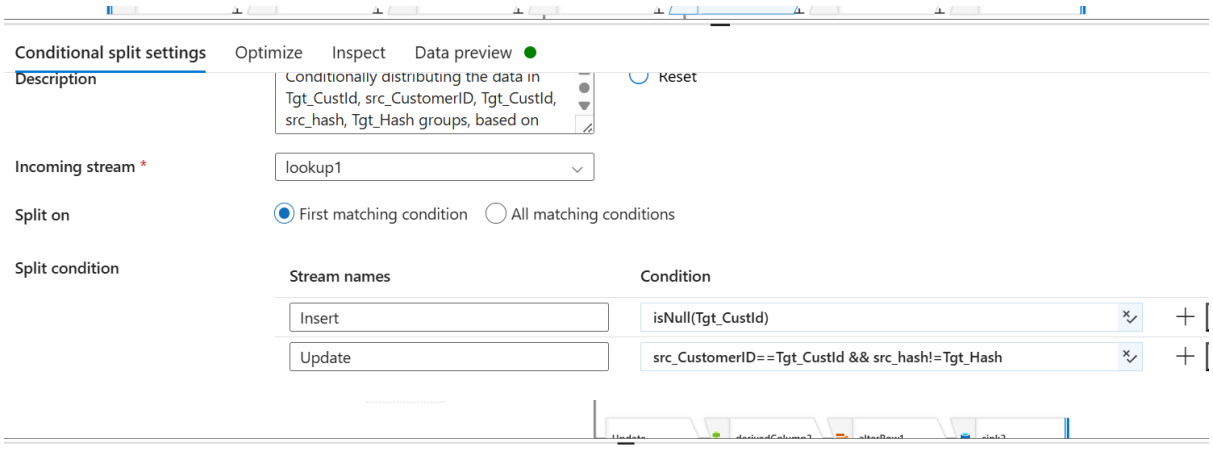


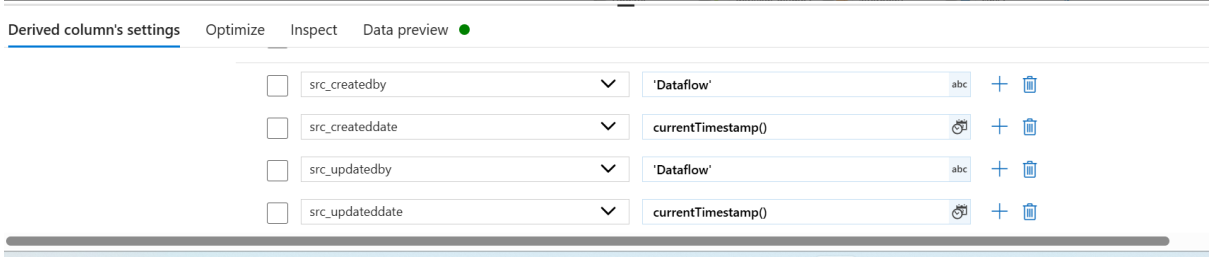
Select:

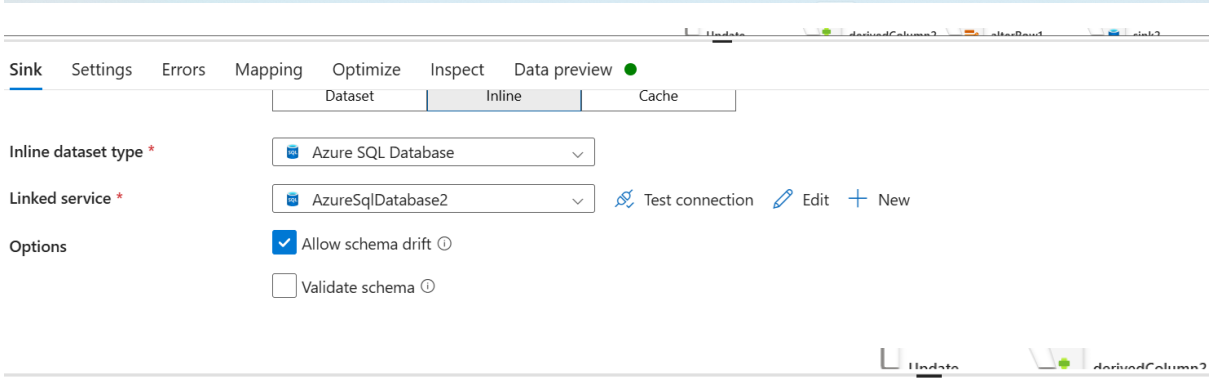


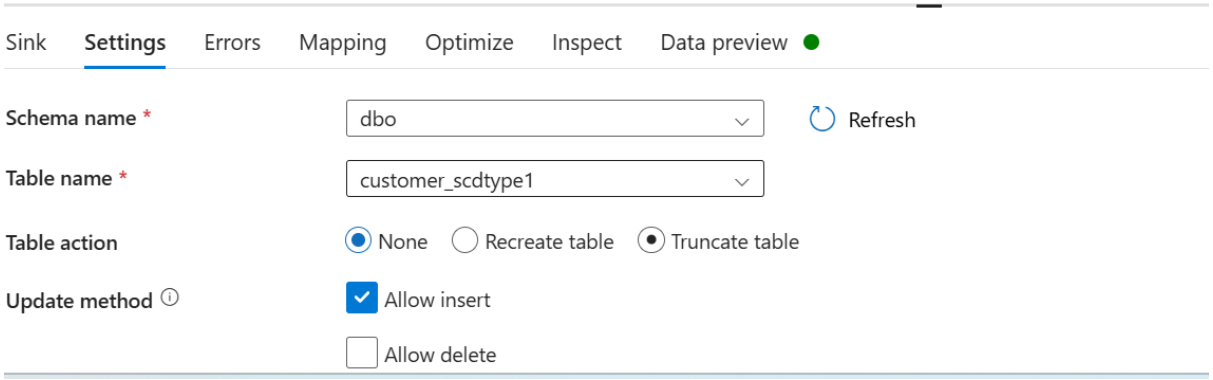




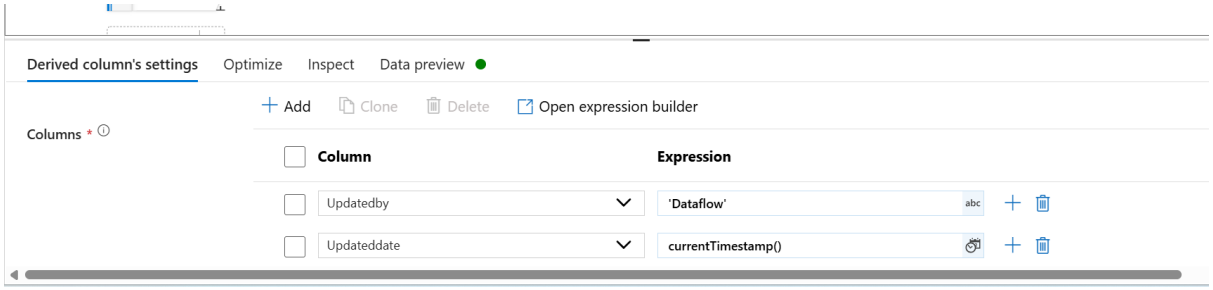

Conditional split settings
Optimize
Inspect
Data preview
Description
Conditionally distributing the data in Tgt_CustId, src_CustomerID, Tgt_CustId, src_hash, Tgt_Hash groups, based on
Reset
Incoming stream *
lookup1
Split on
First matching condition
All matching conditions
Split condition
Stream names
Condition
Insert
isNull(Tgt_CustId)
Update
src_CustomerID==Tgt_CustId && src_hash!=Tgt_Hash

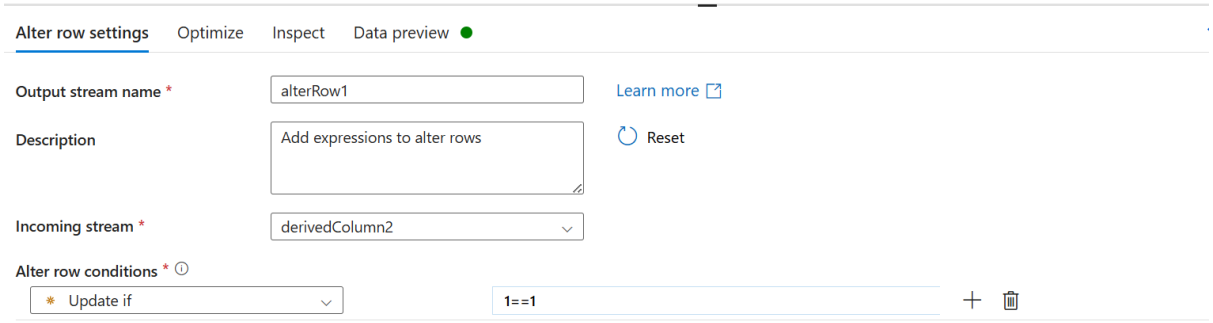

Derived column's settings
Optimize
Inspect
Data preview
src_createdby
'Dataflow'
src_createddate
currentTimestamp()
src_updatedby
'Dataflow'
src_updateddate
currentTimestamp()

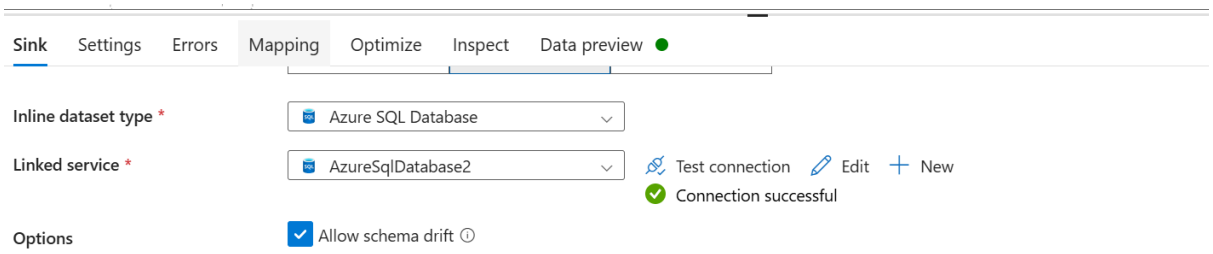

Sink
Settings
Errors
Mapping
Optimize
Inspect
Data preview
Dataset
Inline
Cache
Inline dataset type *
Azure SQL Database
Linked service *
AzureSqlDatabase2
Test connection
Edit
New
Options
Allow schema drift
Validate schema


Sink
Settings
Errors
Mapping
Optimize
Inspect
Data preview
Schema name *
dbo
Refresh
Table name *
customer_scdtype1
Table action
None
Recreate table
Truncate table
Update method
Allow insert
Allow delete

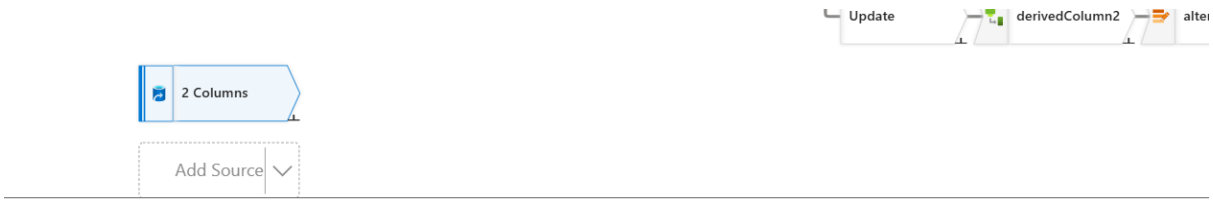
Update:

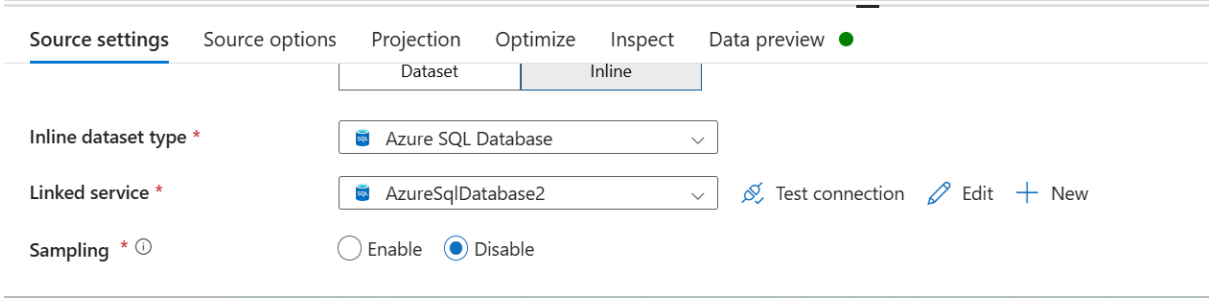






Target:





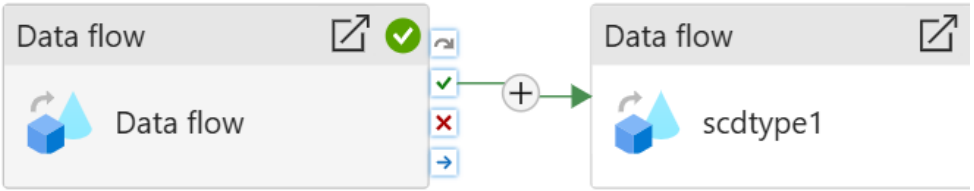
7. Total Pipeline Workflow

The complete pipeline integrates ingestion, transformation, and loading processes. The workflow ensures data consistency, automation, and monitoring using ADF triggers.

- Pipeline Flow:
- Data Ingestion (API / On-Prem)
- Raw Storage in ADLS Gen2

- Data Transformation using Data Flows
- Load to Azure SQL Database

Total Pipeline:

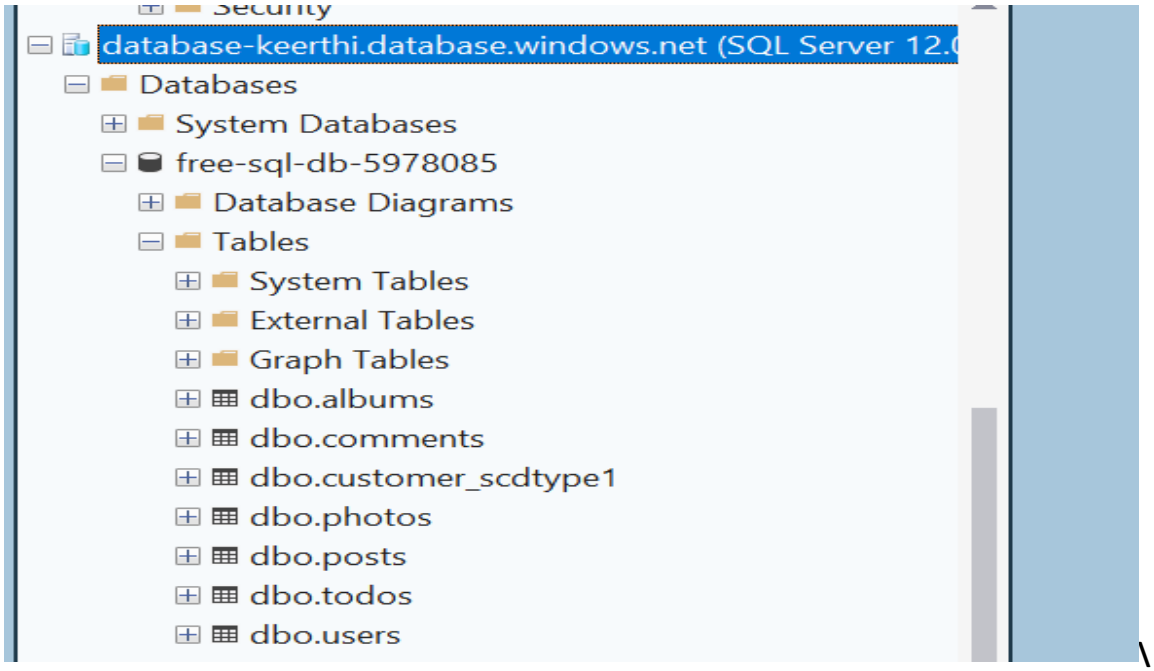


8. Azure SQL Database Configuration

Azure SQL Database is configured as the final structured data repository. Primary keys and indexing strategies are implemented to optimize query performance.

- Features Used:
- Upsert functionality for SCD Type 1
- Data validation constraints

Azure SQL Database:



9. Conclusion

This project focused on implementing a comprehensive data engineering solution that enables data ingestion from multiple sources, including REST APIs and On-Premises systems, into Azure Data Lake Storage Gen2 and Azure SQL Database. Azure Data Factory pipelines were designed to orchestrate

the end-to-end workflow efficiently. Mapping Data Flows were utilized to perform necessary data transformations, cleansing, and business logic implementation. Additionally, a Slowly Changing Dimension Type 1 (SCD Type 1) process was implemented to manage and maintain updated records within the target database. The final solution demonstrates a structured, scalable, and automated approach to building modern cloud-based data integration pipelines.